

The Early Warning System: Predicting Company Bankruptcy



By: Alex, Anik, Daniel, Sanjay
Project Manager: Pritul Vachhani
Team #11

Table of Contents

01

**Project
Overview**

02

**Data
Preprocessing**

03

**Primary Model
Training & Testing**

04

Demo

05

**Challenges and
Learnings**

06

Future Work



01

Project Overview

Project Goal

Our goal is to predict the bankruptcy risk of companies and identify if a company is likely to be 'bankrupt' or 'alive' based on financial features

Dataset Background

- Dataset of **8,262** U.S. public companies (NYSE & NASDAQ), 1999–2018, with complete accounting data.
 - **78,682** firm to year records
- **Dataset Link:**
 - [US Company Bankruptcy](#)

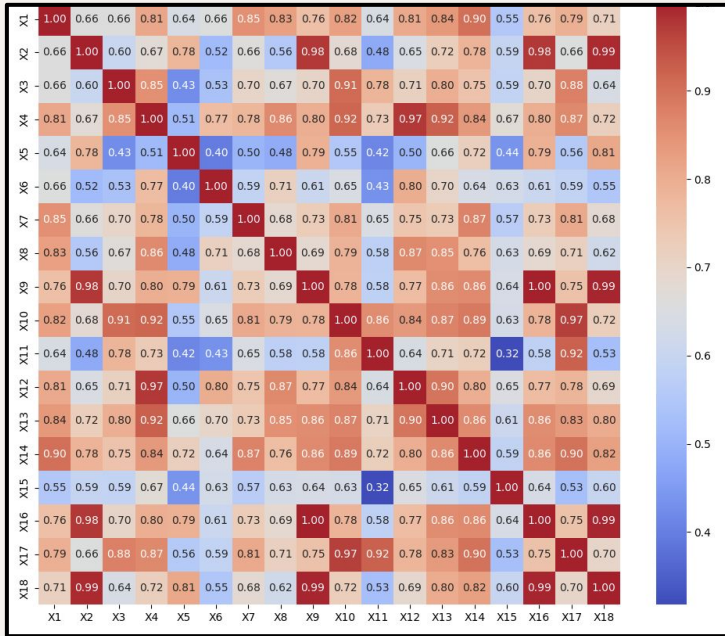


Features Used

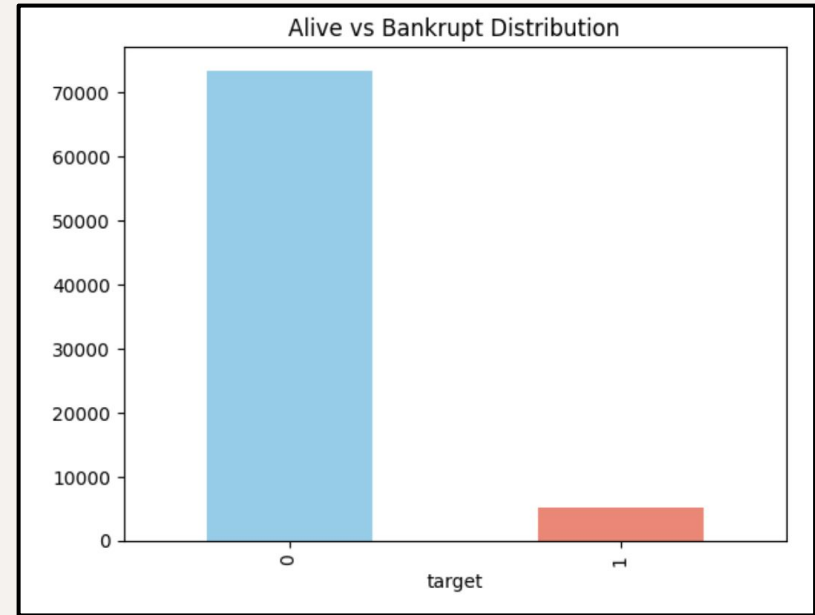
- Dataset uses **18** features:
 - Current Assets
 - Cost of Goods Sold
 - Depreciation and Amortization
 - EBITDA
 - Earnings before interest, taxes, depreciation, and amortization
 - Inventory
 - Net Income
 - Total Receivables
 - Market Value
 - Net Sales
 - Total Assets
 - Total Long-Term Debt
 - EBIT
 - Earnings before interest and taxes
 - Gross Profit
 - Total Current Liabilities
 - Retained Earnings
 - Total Revenue
 - Total Liabilities
 - Total Operating Expenses

Exploratory Data Analysis

Correlation Heatmap



Class Balances



Note: 0 = Alive Companies; 1 = Bankrupt Companies

Tools Used



- Python



- Google Colab



- GitHub



- VS Code



- Scikit-Learn



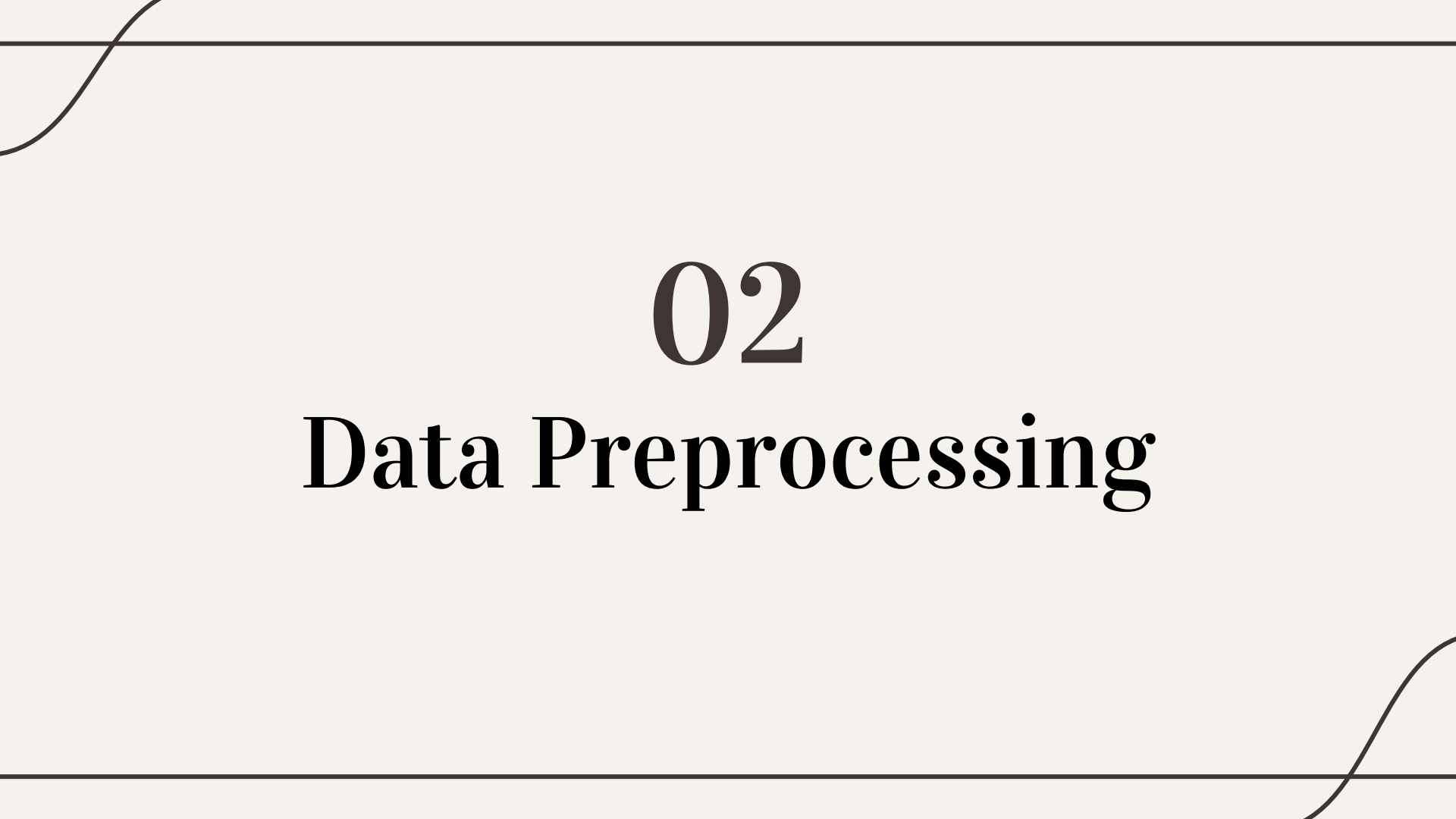
- AWS Elastic Beanstalk



- AWS S3 Bucket



- PostMan



02

Data Preprocessing

Data Preprocessing

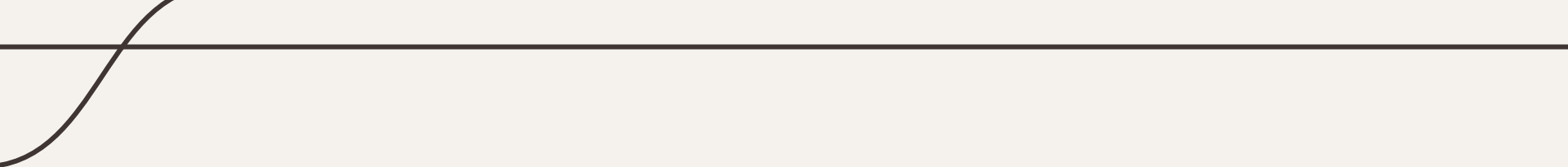
- Dropped three columns
 - Status Label, Company Name, Year
- Dataset was pre-cleaned
 - No missing values
- Kept all outliers in dataset
- Data Splitting By Year:
 - Training (1999-2011)
 - Validation (2012-2014)
 - Testing (2015-2018)

```
df["target"] = (df["status_label"].str.lower() == "failed").astype(int)
features = [c for c in df.columns if c.startswith("X")] # X1..X18

train = df[df["year"].between(1999, 2011)]
val    = df[df["year"].between(2012, 2014)]
test   = df[df["year"].between(2015, 2018)]

# Drop 'company_name' and 'year' after splitting
df = df.drop(columns=['company_name', 'year'])

x_tr, y_tr = train[features], train["target"]
x_va, y_va = val[features],   val["target"]
x_te, y_te = test[features],  test["target"]
```



03

Primary Model Training & Testing Results



Random Forest Model

An ensemble method that combines many decision trees to improve prediction accuracy and reduce overfitting. Chosen for its robustness and strong performance on financial tabular data.

Key Hyperparameters:

- **n_estimators** – number of trees
- **max_depth** – depth of each tree
- **min_samples_split & min_samples_leaf** – prevent overfitting
- **class_weight='balanced'** – handles class imbalance

```
rf = RandomForestClassifier(  
    n_estimators=200,  
    max_depth=15,  
    min_samples_split=20,  
    min_samples_leaf=10,  
    class_weight='balanced',  
    random_state=42,  
    n_jobs=-1  
)  
rf.fit(x_tr, y_tr)
```

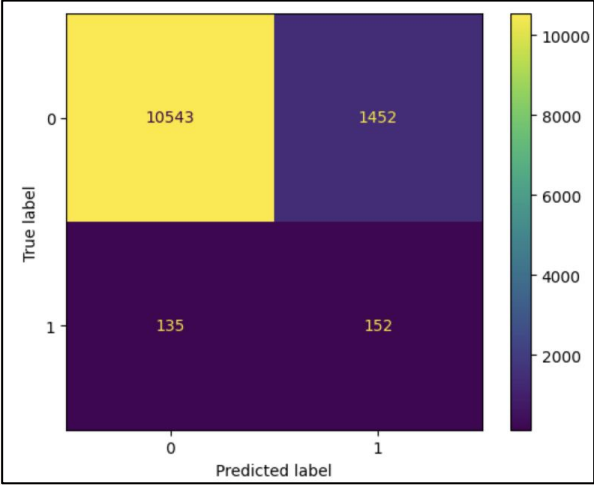
Random Forest Test Results

Classification Report

Class/Average	Precision	Recall	F1-Score	Support
Non Failed	0.99	0.88	0.93	11995
Failed	0.09	0.53	0.16	287
Macro Avg	0.54	0.70	0.55	12282
Weighted Avg	0.97	0.87	0.91	12282

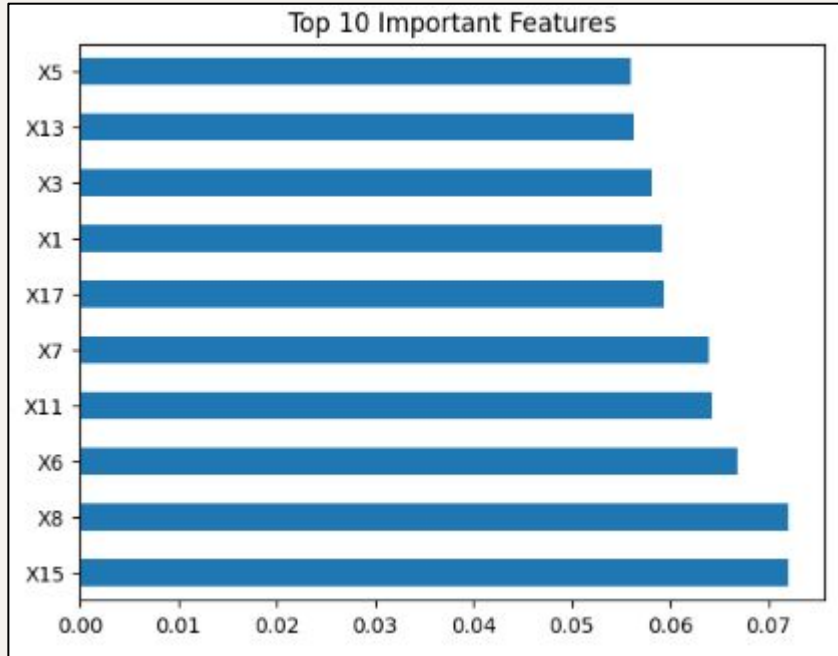
Metric	Value	Total Support (Samples)
Accuracy	0.87 (87%)	12282

Confusion Matrix



Model achieves strong accuracy (87%) and performs very well in identifying Alive companies. Due to significant class imbalance, precision on Failed cases is low, but recall is moderate at 0.53. Meaning the model successfully detects over half of the actual failures. These results indicate the model can be used as an early-warning tool rather than a final decision maker.

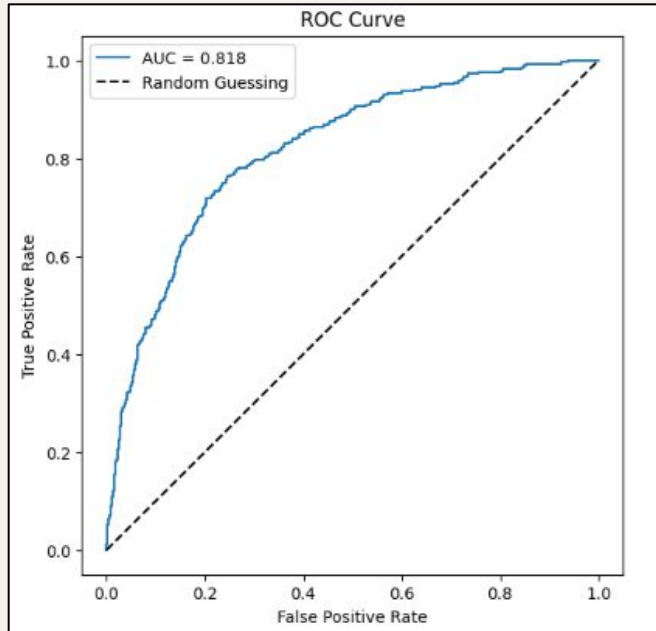
Feature Importance



- Top three factors influencing model's prediction
 - **Retained Earnings (X15)**
 - **Market Value (X8)**
 - **Net Income(X5)**

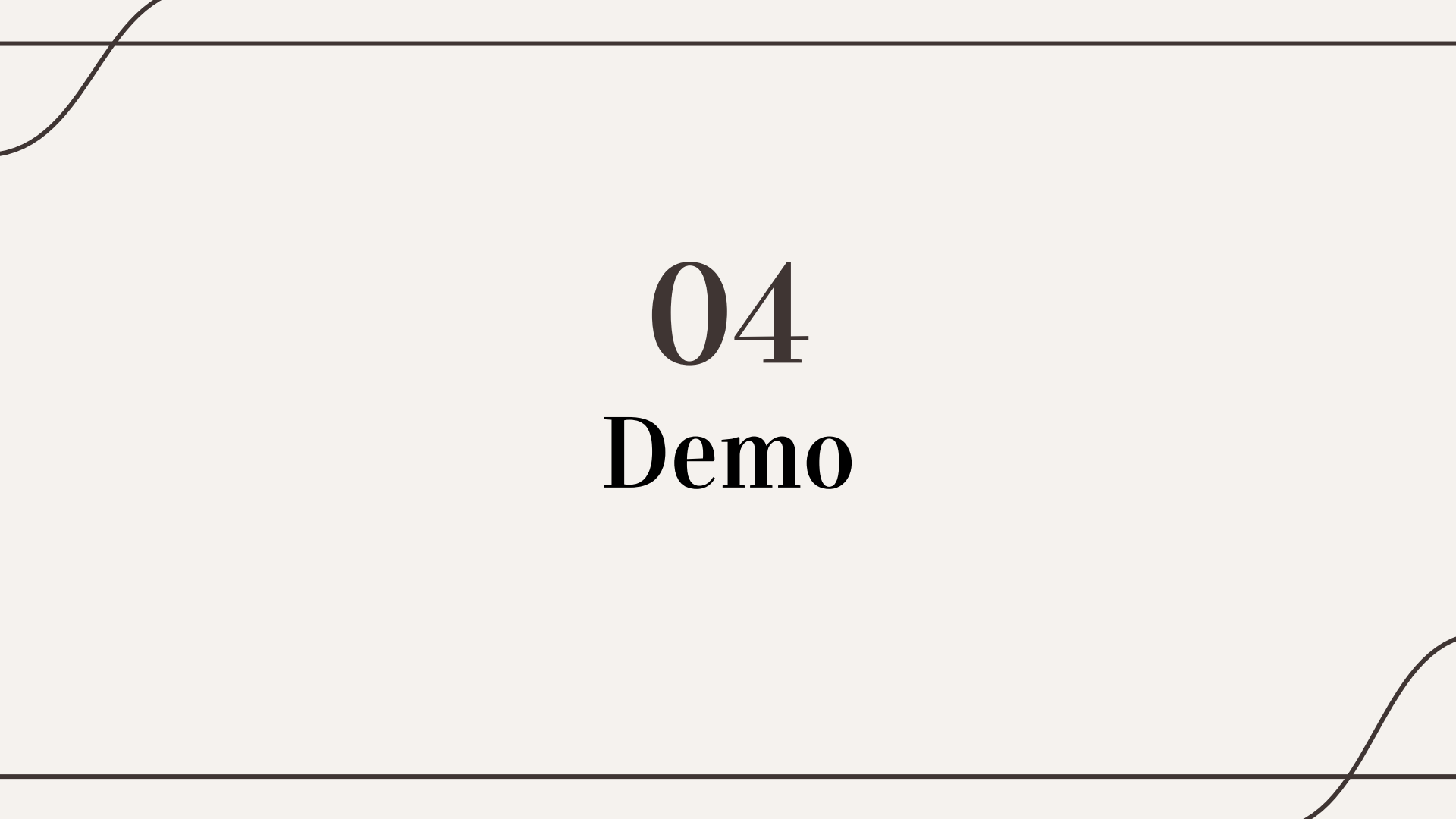
These financial indicators are strongly associated with a company's long-term stability and therefore contribute most to bankruptcy prediction.

ROC Curve – Model Discriminatory Power



Key Insights

- **AUC Score: 0.818**, indicating **strong ability** to distinguish Failed vs Non-Failed companies.
- Model maintains high True Positive Rate across most thresholds.
- Significantly outperforming random guessing (dashed line).
- Robust performance despite **extreme class imbalance**.



04

Demo

Demo

- **Flask**
 - Used for developing web applications
- User Interface
 - 18 Input Fields for all financial parameters with organized grid layout
 - Interactive Info Icons - Click the info icon (i) to see detailed descriptions
 - Real-time Validation - All fields are required and must be numeric
- Local testing in VS Code and deployed on AWS.

Deployment Process

Backend

- Updated ML model → Random Forest
- Resolved Python version mismatch (3.11 → 3.9)
- Rebuilt virtual environment and packaged backend
- Deployed to AWS Elastic Beanstalk
- Tested /predict API in Postman

Frontend

- Updated React files with backend API URL
- Built production bundle (npm run build)
- Hosted frontend on AWS S3
- Verified live web app from S3 URL

QR Code





05

Challenges and Learnings

Challenges

- Class Imbalance(more alive than failure)
- Finding right model
- Balancing between false negative and false positive
- Complexity of model -> Overfitting

Learnings:

- The correct time split for dataset has significant impact on model engine performance.
- High accuracy does not always mean good performance.



06

Future Work

Future Work

- Improve model accuracy for “Bankrupt” companies
- Improve front end
 - Be able to input multiple years of data instead of just one year
- Feature Engineering
- Get more data on bankrupt companies
 - Move towards 50-50 split between bankrupt and non bankrupt companies

The image features a light gray background with two horizontal lines. Each line has a thin, dark brown curved line extending from its left and right ends, creating a decorative frame. The text "Thank You!" is centered between these lines.

Thank You!